COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - DATA STRUCTURE AND ALGORITHM**	**SECTION:**	CPE21S4
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/21/26
NAME:	Del Mundo, Kenneth James C. Mangcucang, Rhianne L	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO.13.1 **PARALLEL ALGORITHMS AND MULTITHREADING**

1. PROCEDURE OUTPUT

PART 1 : SIMPLE ONE-THREADED EXAMPLE

PART 1.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

OUTPUT:

```
Printing integer: 10
Printing string: T.I.P.
```

ANALYSIS:

DEL MUNDO:
The program created one thread, t1, which ran print with 10 and "T.I.P." and displayed both values.

COMPUTER ENGINEERING

QUEZON CITY

Calling join() forced main to wait until t1 finished before returning. Without join(), main could terminate while t1 was still executing, causing the program to crash with an error. The output matches what a standard function call would yield; because only a single thread was used, there was no performance gain. Multithreading in this context helps demonstrate thread execution mechanics rather than computational scaling performance across cores, though not perfectly.

MANGCUCANG:
This part illustrated the basic lifecycle of std::thread. A thread is assigned a target function and its arguments upon instantiation, immediately scheduling it for execution before main waits at join(). The output lines "Printing integer: 10" and "Printing string: T.I.P." printed sequentially because only one thread accessed the console. This example serves to demonstrate thread initialization and synchronization mechanics rather than speed benefits.

PART 2: MULTI - THREADED EXAMPLE

PART2.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {"T.I.P.", "Competent", "Computer", "Engineers"};
    std::vector<std::thread> threads;

    for (size_t i = 0; i < s.size(); i++)
        threads.push_back(std::thread(print, (int)i, s[i]));

    for (auto &th : threads)
        th.join();
    return 0;
}
```

OUTPUT:

```
0 : T.I.P.2 : Computer
1 : Competent
3 : Engineers
```

ANALYSIS:

DEL MUNDO:
The four threads ran at the same time, resulting in an out-of-order execution sequence of 0, 2, 1, 3 instead of 0, 1, 2, 3. Because the OS scheduler decides which thread runs first, creation order does not decide output order, and the sequence can change on every run. The line "0 : T.I.P.2 : Computer" shows two threads writing to std::cout at once, causing their output to get mixed together. Locking a std::mutex around the print call would fix this and keep the lines clean.

MANGCUCANG:
Our output came out of order with garbled text, which shows that threads are non-deterministic and that std::cout is a shared resource requiring synchronization. Every thread called print at almost the exact same moment, so one thread's text landed right in the middle of another's line. Since each task only prints one line, creating four threads costs more than the work itself. Multithreading would only help here if each task were much heavier.

SUPPLEMENTARY:

PART A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

1. Write a definition of multithreading and its advantages/disadvantages.

Multithreading allows a program or operating system to run multiple tasks at the same time. This means an application can handle several requests or serve multiple users at once without needing to launch separate copies of itself.

The advantage here is performance and responsiveness. By pushing heavy background tasks to a secondary thread, the main user interface stays smooth and snappy. It also lets you make full use of modern multi-core processors, scaling up performance almost linearly across cores while squeezing extra efficiency out of idle time when a thread is stuck waiting on disk or network responses.

That speed and flexibility comes with disadvantages. When multiple threads try to write to the same memory location at once, you run into race conditions that can corrupt your data. Threads can also end up in deadlocks, effectively freezing up because each one is waiting on a lock the other holds. To make

matters worse, these bugs depend entirely on precise timing, making them tricky to spot during testing and notoriously frustrating to reproduce.

2. Rationalize the use of multithreading by providing at least 3 uses-cases.

Multithreading is a software design approach that divides an application into multiple concurrent threads of execution to maximize processor efficiency, improve overall system throughput, and prevent user interfaces from freezing.

In high-concurrency environments like web servers and API gateways, multithreading allows the system to process other incoming client requests on secondary threads while primary threads wait on slow network or database operations.

For desktop and mobile applications with graphical interfaces, offloading heavy computational tasks to background worker threads ensures that the main event thread remains free to render frames smoothly and respond immediately to user inputs.

In computationally demanding applications such as 3D rendering engines and physics simulators, multithreading splits massive datasets across multiple physical CPU cores to run calculations simultaneously, dramatically cutting processing times.

3. Differentiate between parallelism and concurrency.

Concurrency is about dealing with many things at once. It is a structural property where multiple tasks make progress over overlapping time periods, often by interleaving execution on a single core via rapid context switching.
Parallelism is about doing many things at once. It is an execution property where multiple tasks or computations run at the exact same physical instant, which requires multiple processing units (such as multi-core CPUs or GPUs).
Pros and Cons of Multithreading: Advantages and Disadvantages
Difference between Concurrency and Parallelism - GeeksforGeeks

PART B: Create C++ Code and show a solution that satisfies the given requirements below

COMPUTER ENGINEERING

QUEZON CITY

```cpp
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> g{0};

void add(int n) {
    g += n;
}

int main() {
    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    std::cout << "Display (before any join): " << g << std::endl;
    t1.join();
    std::cout << "Display (after T1.join):   " << g << std::endl;
    t2.join();
    std::cout << "Display (after T2.join):   " << g << std::endl;
    t3.join();
    std::cout << "Display (after T3.join):   " << g << std::endl;
    return 0;
}
```

OUTPUT:

```
Display (before any join): 0
Display (after T1.join):   60
Display (after T2.join):   60
Display (after T3.join):   60
```

ANALYSIS:

DEL MUNDO:

Our specific run printed 0 before any join() calls, followed by 60 after each of T1.join(), T2.join(), and T3.join(). While early values can vary depending on thread execution timing, our run showed 0 initially because the main thread printed before worker threads completed their additions. After T1.join(), we saw 60 because the add function is so short that T2 and T3 had already finished executing, even though T1.join() technically only guarantees T1's completion. The final value of 60 after T3.join() is the only value guaranteed by synchronization. We used std::atomic<int> so that three threads adding to one global do not cause a data race; a std::mutex with std::lock_guard would also work.

MANGCUCANG:

When you construct a thread in C++, it starts executing immediately. That's why displaying the shared total early on will often catch it mid-calculation. Calling T1.join() only confirms that the first thread has finished; it doesn't tell you anything about whether T2 or T3 have finished or even started running yet. You can only count on that final value of 60 once all three threads have been joined. Now, because add is such a quick operation, all three threads might finish before you even reach the first print statement—making it look like 60 is printed every time—but relying on that timing is a classic trap.

I chose std::atomic<int> to prevent a data race. Having three threads write to a standard int at the same time triggers undefined behavior, as the handout points out. Non-atomic operations like g += n are actually multi-step read-modify-write sequences under the hood, meaning threads can easily overwrite each other's changes and lose updates. Wrapping the critical section in a std::mutex with

std::lock_guard would be another completely valid fix here, but atomics keep simple numeric updates straightforward and efficient.

PART C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <random>
#include <algorithm>

// Standard Merge Step
void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int p = 0; p < k; ++p) {
        arr[left + p] = temp[p];
    }
}


void mergeSortSequential(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;

    mergeSortSequential(arr, left, mid);
```

```
    mergeSortSequential(arr, mid + 1, right);
    merge(arr, left, mid, right);
}


void mergeSortParallel(std::vector<int>& arr, int left, int right, int depth) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;

    if (depth < 3) {
        std::thread t1(mergeSortParallel, std::ref(arr), left, mid, depth + 1);
        std::thread t2(mergeSortParallel, std::ref(arr), mid + 1, right, depth + 1);
        t1.join();
        t2.join();
    } else {
        mergeSortSequential(arr, left, mid);
        mergeSortSequential(arr, mid + 1, right);
    }

    merge(arr, left, mid, right);
}


int main() {
    int N = 1000000;
    std::vector<int> arr1(N), arr2(N);

    std::mt19937 rng(42);
    for (int i = 0; i < N; ++i) {
        int val = rng() % 1000000;
        arr1[i] = val;
        arr2[i] = val;
    }


    auto startParallel = std::chrono::high_resolution_clock::now();
    mergeSortParallel(arr1, 0, N - 1, 0);
    auto endParallel = std::chrono::high_resolution_clock::now();

    auto startSeq = std::chrono::high_resolution_clock::now();
    mergeSortSequential(arr2, 0, N - 1);
    auto endSeq = std::chrono::high_resolution_clock::now();

    auto parallelTime = std::chrono::duration_cast<std::chrono::milliseconds>(endParallel - startParallel).count();
```

```
auto seqTime = std::chrono::duration_cast<std::chrono::milliseconds>(endSeq - startSeq).count();

std::cout << "Sequential Merge Sort Time: " << seqTime << " ms\n";
std::cout << "Parallel Merge Sort Time:   " << parallelTime << " ms\n";

return 0;
```

ANALYSIS:

DEL MUNDO:

We sorted 1,000,000 random integers with Merge Sort. The sequential version took 162.0 ms and the multithreaded version took 51.5 ms, a speedup of about 3.14x. The multithreaded version splits the array in separate threads down to depth < 3, which creates 14 threads in total and leaves 8 sub-arrays to be sorted sequentially. The speedup is lower than the thread count because the merges at the top levels remain sequential and because creating and scheduling threads adds overhead.

MANGCUCANG:

The multithreaded version is faster because the left and right halves of the array are independent, so threads can sort them at the same time. No mutex was needed because each thread works only on its own range, and join() makes sure both halves are sorted before merge runs. Limiting the depth with if (depth < 3) stops the program from creating too many threads—if it created a thread for every recursive call, the overhead would be larger than the benefit. On a small array, we would expect the sequential version to be faster. Overall, I think I did well in this activity, and my area for improvement is debugging subtle race conditions and choosing the right number of threads for my CPU.

CONCLUSION:

DEL MUNDO:
In this activity, I learned how to use std::thread in C++ to execute functions concurrently and how .join() forces the main thread to pause until a worker thread completes its task. In Part 2, the output order was not identical to the vector order, clearly demonstrating that execution timing is determined non-deterministically by the operating system scheduler rather than the order of thread creation. In Part B, three threads concurrently modified a single global variable, showing unpredictable values before and after each .join() call until T3.join() finally guaranteed the correct total of 60—reinforcing that shared memory operations strictly require primitives like std::atomic or std::mutex to prevent data races. In Part C, the multithreaded Merge Sort completed in 51.5 ms compared to 162.0 ms for the sequential implementation, achieving a speedup of about 3.14x; this gain was smaller than the total core count because top-level merges remain sequential and thread creation adds fixed system overhead. Overall, I think I did well in this activity, and my main area for improvement is debugging subtle race conditions and choosing the optimal thread depth for a given CPU architecture.

MANGCUCANG:
This activity showed me the difference between concurrency, where tasks overlap in time, and parallelism, where tasks run at the same instant on multiple cores. Running the examples taught me that threads have a cost, so for tiny tasks like printing one line, creating a thread takes more effort than the work itself. Multithreading only paid off in Part C, where the left and right halves of the merge sort were independent, so no locks were needed and the program ran faster. Limiting the thread depth to 4 kept the number of threads manageable, because spawning a thread for every recursive call would have made the program slower.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:
Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

COMPUTER ENGINEERING
QUEZON CITY

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.